

RAG (Retrieval Augmented Generation)

Part - 1

1. What is RAG?

- RAG combines the power of LLMs with external knowledge.
- Instead of relying only on the training data, LLMs retrieve relevant information from an external source (documents) and use it to generate accurate, context-aware answers.

2. Why RAG?

- LLMs have limited knowledge (cut-off date).
- Can hallucinate or give incorrect answers.
- RAG provides up-to-date and domain-specific information.
- Improves accuracy, reliability and reduces hallucinations.

3. Core Idea

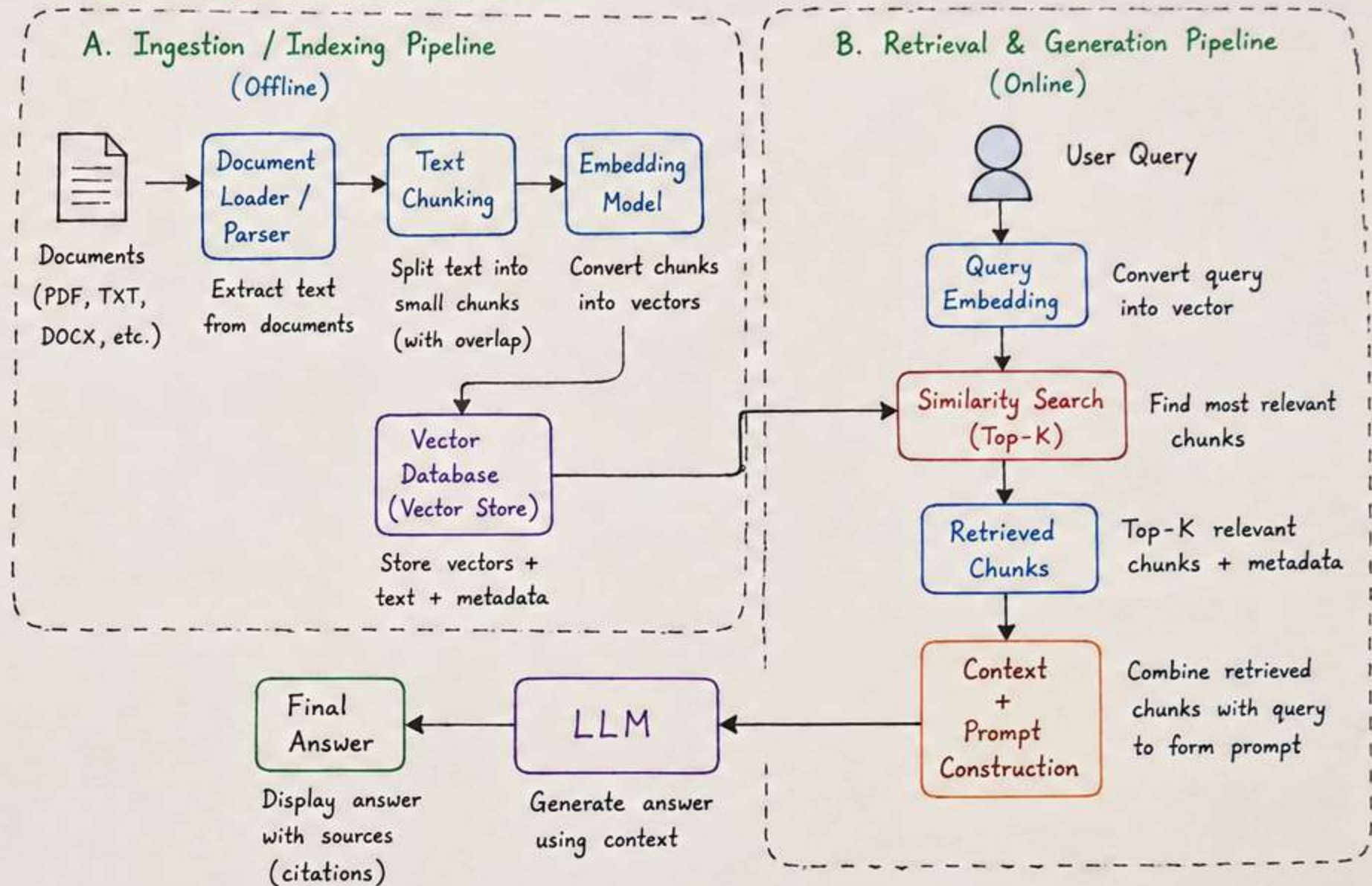
Retrieve relevant information →
Provide it to LLM → Generate Answer

4. RAG Architecture (High Level)

RAG works in two main pipelines:

- 1) Ingestion / Indexing Pipeline (offline)
- 2) Retrieval & Generation Pipeline (online)

5. RAG Architecture Diagram



6. Key Components

1. Document Loader / Parser → Reads and extracts raw text from files.
2. Text Chunking → Splits text into manageable chunks (size + overlap).
3. Embedding Model → Converts text chunks & query into numerical vectors.
4. Vector Database (Vector Store) → Stores vectors along with original text & metadata.
5. Similarity Search → Finds top-K relevant chunks based on vector similarity (e.g., cosine similarity).
6. Prompt Construction → Combines retrieved context with user query.
7. LLM (Generator) → Generates final answer using the provided context.
8. Sources / Citations → Shows from which documents/chunks the information is taken.

Popular Tools

- LangChain
- FAISS / Chroma / Pinecone (Vector DBs)
- OpenAI / Gemini / Llama (LLMs)
- Streamlit / Gradio (for UI)

7. Detailed Explanation of Each Step

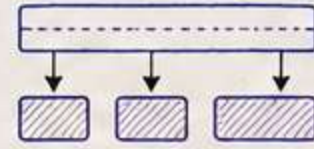
① Document Loader / Parser

Loads documents from various sources (PDF, TXT, DOCX, etc.)
Converts them into raw text.



② Text Chunking

Splits large text into smaller chunks.
Each chunk is a meaningful piece of text.
Overlap ensures context is not lost between chunks.



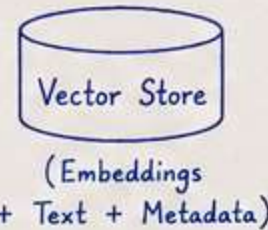
③ Embedding Model

Converts each text chunk into a dense vector
(numerical representation of meaning).

[0.12, -0.34, 0.56, ...]

④ Vector Database (Vector Store)

Stores the embeddings along with:
• Original text (chunk)
• Metadata (source, page no., chunk id, etc.)
Enables fast similarity search.



⑤ Query Embedding

Converts the user question into a vector
using the same embedding model.

[0.21, -0.18, 0.61, ...]

⑥ Similarity Search (Top-K Retrieval)

Compares query vector with all stored vectors
using similarity metric (cosine similarity).
Returns Top-K most similar chunks.

Top-K Results

1. Chunk 17	0.92
2. Chunk 83	0.87
3. Chunk 31	0.84
...	...

⑦ Context + Prompt Construction

Combines the retrieved chunks and places them
in a prompt along with the user question
and instructions for the LLM.

Prompt =
System Instructions
+
Context (Top-K Chunks)
+
User Question



LLM

⑧ LLM (Generation)

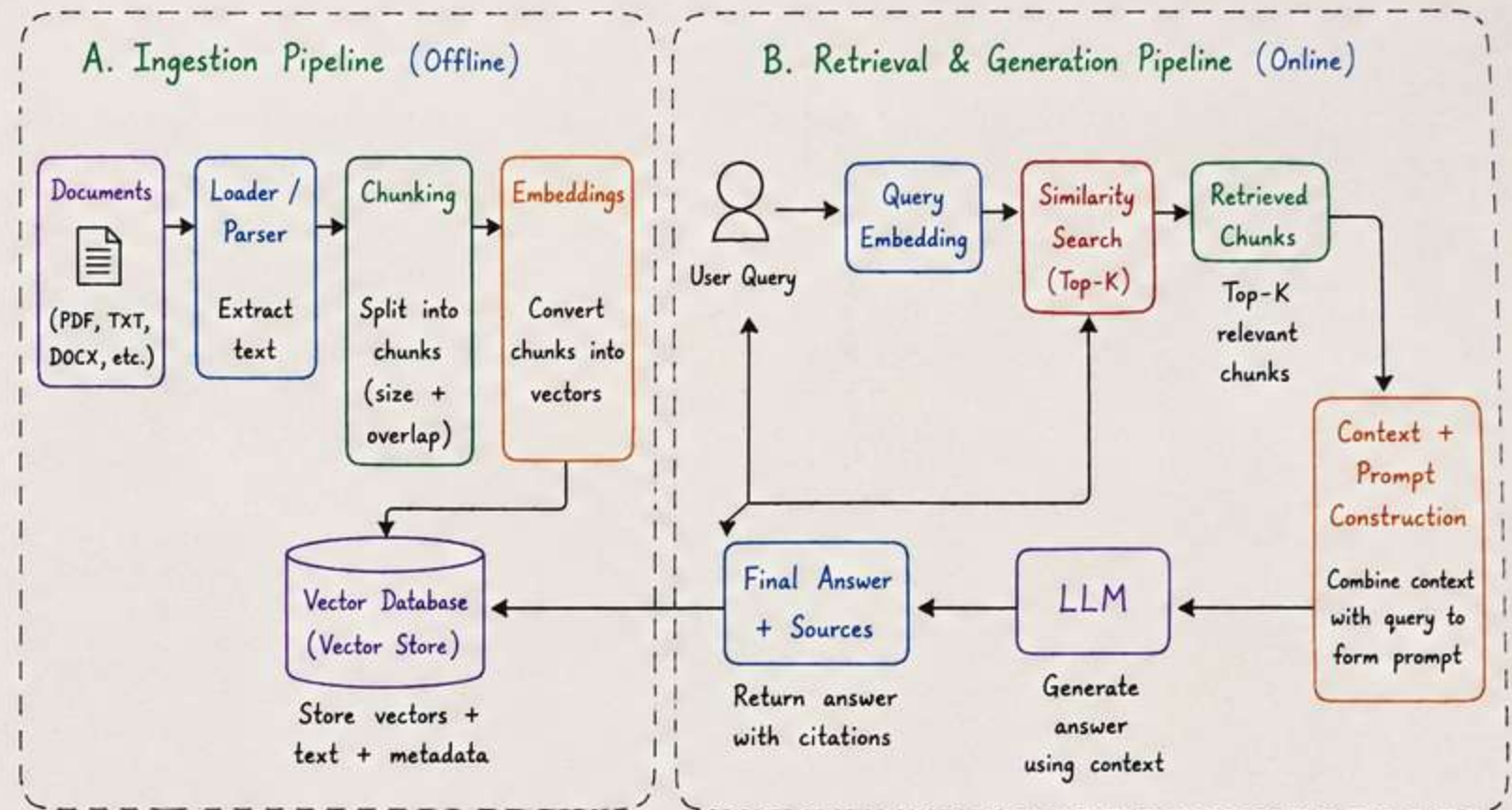
LLM reads the context and user question,
understands it, and generates a grounded,
evidence-based answer.

⑨ Final Answer + Sources

The answer is returned to the user along
with the sources (document name, page no.
or chunk id) for transparency.

Answer
.....
Sources:
• Doc.pdf (Page 42)
• Doc.pdf (Page 43)

8. End-to-End RAG Pipeline (Summary)



9. Important Points to Remember

- Use the same embedding model for documents and queries.
- Chunk size & overlap greatly affect retrieval quality.
- Top-K value: small K → less context, large K → more noise.
- Quality of retrieved context = quality of final answer.
- RAG reduces hallucinations but does not eliminate them completely.
- Always show sources → builds trust and transparency.

Golden Rule

Garbage in →
Garbage out.
Better data & better
retrieval →
Better answers.

10. Basic Tech Stack for a First RAG App (Example)

- | | |
|--|--|
| • Language : Python | • LLM : OpenAI / Gemini / Llama (via API) |
| • Document Loader : PyMuPDF / pdfplumber | • Framework (optional) : LangChain (later) |
| • Text Splitter : RecursiveCharacterTextSplitter | • UI : Streamlit (simple & fast) |
| • Embeddings : sentence-transformers | • Others : tiktoken, python-dotenv, etc. |
| • Vector Store : FAISS / Chroma | |



Start Simple, Understand Deeply, Then Scale Gradually! 😊

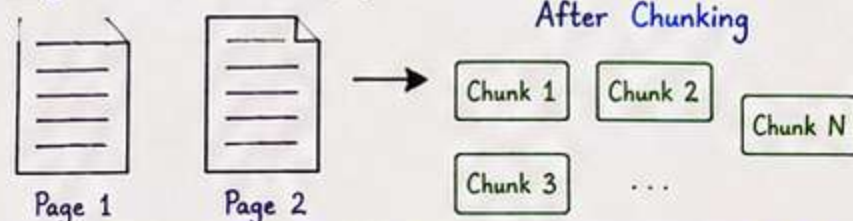
CHUNKING & EMBEDDINGS

1. What is a Chunk?

- A chunk is a small, manageable piece of text split from a larger document.
- Each chunk is treated as an independent unit for embedding and retrieval.
- Good chunking = Better retrieval quality.

Example

Original Document (2 pages)



2. Chunk Size

- If chunk size is too small → loses context.
- If chunk size is too large → less precise retrieval.
- Typical range: 300 - 1000 tokens (experiment and choose what works best)

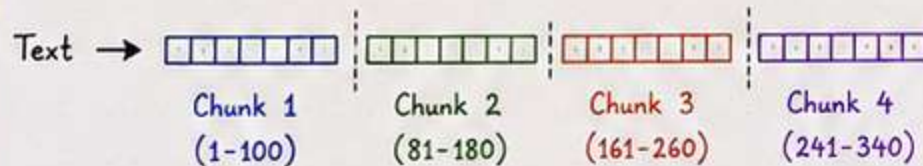
3. Chunk Overlap

- Overlap helps in maintaining context between chunks.
- Prevents important information from being cut in between.
- Typical overlap: 10% - 20% of chunk size

Example

Chunk Size = 100 tokens

Overlap = 20 tokens



(Overlap of 20 tokens between consecutive chunks)

4. Types of Chunking

① Fixed-Size Chunking (Simple & Common)

- Splits text into chunks of fixed size with overlap.
- Easy to implement.

② Semantic Chunking (Advanced)

- Splits based on meaning / topics / sentences.
- Keeps semantically related content together.
- Better for long, complex documents.

For a basic RAG app → Start with Fixed-Size Chunking, then try Semantic Chunking.

5. Tokens vs Characters

- LLMs understand text in terms of **tokens**, not characters.
- 1 token $\approx$ 3 to 4 characters in English.
- Always consider token count while chunking.

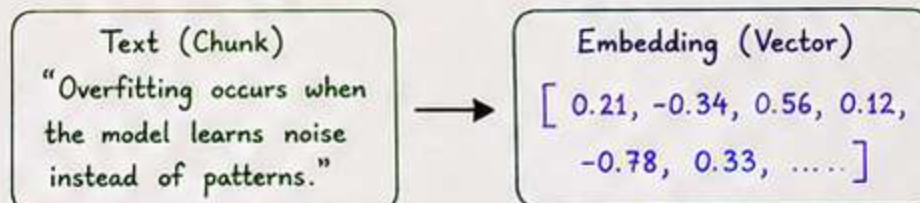
Example

Text: "RAG is a powerful technique for LLM applications."

Characters	Tokens (approx)
52	13

6. What is an Embedding?

- An embedding is a **numerical vector** representation of text.
- It captures the **semantic** meaning of the text.
- Similar meaning → Similar vectors.
- Different meaning → Distant vectors.



7. Embedding Dimensions

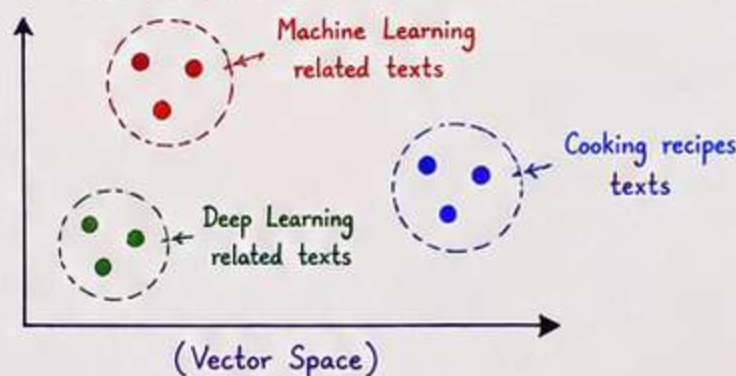
- Embeddings are high-dimensional vectors.
- Dimension depends on the embedding model.

Model	Dimension (Length of Vector)
OpenAI text-embedding-3-small	1536
... text-embedding-3-large	3072
sentence-transformers (all-MiniLM-L6-v2)	384
BAAI/bge-small-en-v1.5	384
intfloat/multilingual-e5-base	768

(Higher dimension → More expressive representations)

8. Why Similar Meaning → Nearby Vectors?

- Embedding models are trained on large text data.
- They learn to place semantically similar text close to each other in the vector space.
- This allows us to find relevant information even if the exact words are different.



Key Idea

Meaning is captured by position in vector space, not by individual words.

Remember

Good chunking + Good embeddings = High quality retrieval = Better RAG answers 😊

1. What is a Vector Database?

- A vector database stores, indexes and retrieves high-dimensional **vectors** along with their **metadata**.
- Optimized for **similarity search** (nearest neighbor search).
- Essential for RAG to find semantically similar chunks.

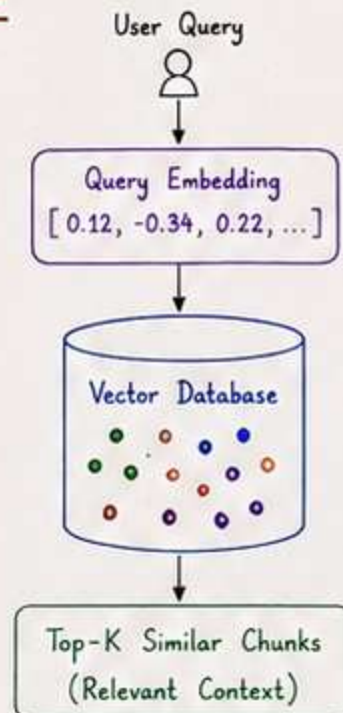
2. What is Stored in a Vector DB?

ID	Vector (Embedding)	Text (Chunk)	Metadata
chunk_001	[0.21, -0.34, 0.56, ...]	RAG combines LLMs ...	source: doc.pdf page: 12
chunk_002	[-0.18, 0.61, -0.22, ...]	Chunking helps in ...	source: doc.pdf page: 13
chunk_003	[0.33, 0.12, -0.78, ...]	Vector DB enables fast ...	source: doc.pdf page: 14
...	...	...	...

(Vector + Original Text + Metadata)

3. How Similarity Search Works?

- User asks a question.
- Query is converted into embedding (vector).
- Query vector is compared with all stored vectors in DB.
- DB returns Top-K most similar vectors (chunks).



5. Top-K Retrieval

- After similarity search, we take Top-K most similar chunks.
- K is a hyperparameter (commonly 3, 5, 10...).
- Larger K → More context but may introduce noise.
- Smaller K → Less context but more focused.

Example

Query:
"How does regularization
help in reducing overfitting?"

Top-3 Retrieved Chunks

- Chunk 17 (Score: 0.91)
- Chunk 83 (Score: 0.87)
- Chunk 31 (Score: 0.84)

4. Similarity Metrics

Metric	Formula (for vectors A, B)	Notes
Cosine Similarity	$\frac{A \cdot B}{\ A\ \ B\ }$	Most commonly used in RAG.
Dot Product	$A \cdot B$	Good when vector magnitudes are meaningful.
Euclidean Distance	$\ A - B\ _2$	Lower distance = More similar.

★ Cosine Similarity Range: -1 (least similar) to 1 (most similar)

Key Idea: We want the highest similarity (close to 1) or lowest distance.

8. Why Retrieval May Return Irrelevant Chunks?

- Embeddings may not perfectly capture meaning.
- Different words, same meaning → distance may be large.
- Important info may be in a different chunk.
- Noisy or too large chunks add unrelated context.
- Poor quality documents / OCR errors.

Example

Query:
"Benefits of dropout"

Retrieved (irrelevant):

- History of dropout in schools
- How to remove duplicates in pandas



6. Metadata Filtering (Optional but Useful)

- Filter chunks using metadata before or after retrieval.
- Example: Filter by document name, page range, date, tags, category.

Example

Filter: source = "doc.pdf" AND page >= 10 AND page <= 20
→ Search happens only on filtered subset.

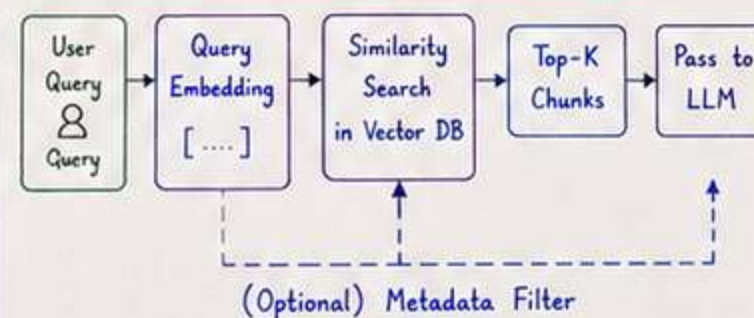
Why Useful?

- Reduces search space
- Improves relevance
- Supports multi-tenant or domain-specific data

7. Retrieval vs Ranking vs Reranking

- Retrieval** → Get initial Top-K chunks using similarity search.
- Ranking** → Order chunks based on similarity score.
- Reranking** → Use a more powerful model (cross-encoder / LLM) to re-score and reorder the retrieved chunks. (Improves final relevance)

9. Basic Retrieval Pipeline (Summary)



10. Popular Vector Databases

- FAISS (Meta)
- Chroma (Open Source)
- Pinecone (Managed)
- Weaviate (Open Source)
- Milvus (Open Source)
- Qdrant (Open Source)



Remember: Good retrieval is the foundation of good RAG. If retrieval is poor, generation will also be poor.



1. Context Window (LLM Limit)

- LLMs have a maximum context length (tokens).
- The prompt = System + Context + Query must fit within this limit.
- If context is too large → Important info may get truncated.
- Keep retrieved context short, relevant and well-structured.

Example (GPT-4o context window ≈ 128K tokens)

System	Retrieved Context	User Query	Answer
500	4000	100	500

Total used = 5100 tokens (within limit)

2. Prompt Construction (RAG Prompt)

- Good prompt design improves accuracy and reduces hallucinations.

Typical RAG Prompt Template

System:

You are a helpful assistant. Answer the question using only the provided context.
If the answer is not present in the context, say "I don't have enough information."
Do not make up information.

Context:

{retrieved_context}

Question:

{user_query}

Answer:

3. Grounding

- Grounding means the answer is based on the provided context (not on the model's own knowledge).
- The more relevant and accurate the context, the better the grounding.

Goal

LLM should "stick to the context" and avoid using external knowledge.



4. Hallucination in RAG

- Hallucination = LLM generates information that is not in the context.

Causes:

- Irrelevant / noisy context
- Not enough relevant context
- Ambiguous query
- Weak prompt
- Model tries to "fill the gap" with its own knowledge

Example

Query: "What is the capital of France?"

Context: (contains anything else)

LLM might still answer "Paris" → Hallucination!



5. How to Reduce Hallucinations

- Use strict prompt instructions.
- Retrieve high-quality, relevant context.
- Provide sufficient Top-K context.
- Use metadata filtering.
- If answer not present → say "I don't have enough information."
- Show sources to increase transparency.
- Keep temperature low (e.g., 0-0.2).

★ There is no 100% way to eliminate hallucinations, but we can reduce them significantly.

6. What if Answer is Not in Context?

- The system should not guess.
- Follow the prompt and respond:

"I don't have enough information in the provided context."

- This builds trust and prevents wrong information.



7. Source / Citation

- Always show from which document or chunk the information is taken.
- Helps users verify and increases trust.

Example Answer

Answer: Overfitting can be reduced using regularization, dropout, and more data.

Sources:

- ML_Notes.pdf - Page 42
- ML_Notes.pdf - Page 43
- DeepLearning.pdf - Page 101



8. Evaluating RAG (Retrieval Quality)

- We evaluate retrieval first (before generation).

Metric	What it Measures	Good When...
Precision@K	How many retrieved chunks are actually relevant	High value (close to 1)
Recall@K	How many relevant chunks are successfully retrieved	High value (close to 1)
MRR	Quality of ranking (how early relevant chunk appears)	High value
Hit Rate@K	At least one relevant chunk is in Top-K	High value
NDCG@K	Considers ranking position and relevance	High value

★ Good retrieval = High Precision + High Recall

9. End-to-End Quality

- Even with good retrieval, generation can fail.
- So evaluate the final answer too.

Metrics for final answer:

- Faithfulness → Is answer grounded in context?
- Answer Relevance → Is it relevant to query?
- Correctness → Is it factually correct?
- Helpfulness → Is it useful to the user?

A good RAG system =
Good Retrieval + Good Generation
+ Good Prompting



10. Basic Limitations of RAG (V1)

- Depends on quality of documents.
- Struggles with very long / complex queries.
- May miss information if it is not in documents.
- Retrieval can return irrelevant chunks.
- Performance depends on chunking strategy.
- Cannot do reasoning beyond provided context.
- Not suitable for real-time / dynamic data (unless data is re-indexed).

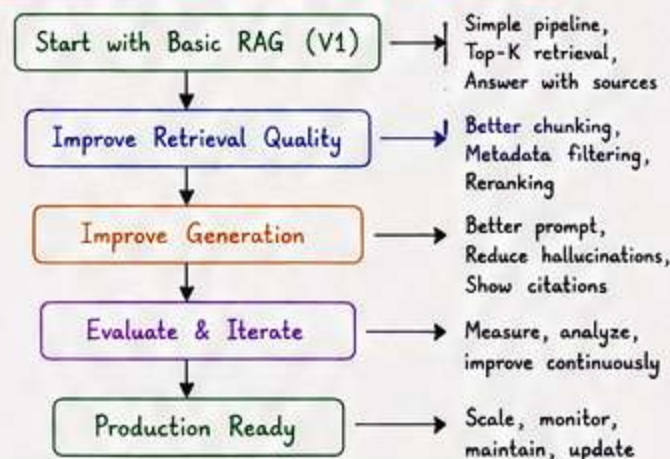


11. How to Improve (V2 and Beyond)

- Better chunking (semantic / hierarchical)
- Hybrid search (vector + keyword)
- Reranking with cross-encoder
- Query rewriting / multi-query retrieval
- Parent-child retrieval
- Metadata filtering
- More powerful LLM / fine-tuned models
- Better prompt engineering
- Evaluation & continuous improvement



12. RAG Development Roadmap (High Level)



Key Takeaways



- ★ RAG is only as good as the context it retrieves.
- ★ Good retrieval reduces hallucinations.
- ★ Good prompt keeps the model grounded.
- ★ Always show sources.
- ★ If not in context → Don't guess.
- ★ Measure, improve, and iterate.

Retrieve the right context,
Guide the model well,
Get the right answer!



Build simple first, understand deeply, then optimize and scale.

